

ЖУРНАЛ РАЗРАБОТКИ

Сервис прогнозирования инцидентов в инженерных коллекторах

Задача №8 «Город»

Конкурс «Лидеры цифровых трансформаций 2026»

Постановщик: Департамент жилищно-коммунального хозяйства города Москвы

Эксплуатант: АО «Москоллектор»

Команда «Дискреция творца»

Москва, сентябрь 2026

Содержание

День 1 — данные, контракт, триты	3
Сделано	3
Витрины	4
Покрытие контракта — 100 %	4
Найденные дефекты	4
Журнал событийный, а не телеметрический	4
Каналы-тёзки: результат однозначный	4
Логические домены: половина сети слепа	5
Дальше — День 2	5
День 2 — признаки, модель M1, объяснимость	5
Сделано	5
Таргет	5
Ошибка, пойманная на проверке баланса	6
Объяснимость: обещание выполнено буквально	6
Что предсказывает модель	6
Горизонт: где проходит граница возможного	6
Дальше — День 3	7
День 3 — сквозной срез	7
Контрольная точка взята	7
Сделано	7
Разделение хранилищ	7
Решение по образу	7
Оперативный контур	7
Найденные дефекты	8
Пример вердикта из работающей системы	8
Дальше — День 4	8
День 4 — подтопление, пожар, линейная схема	8
Сделано	8
M2 «Подтопление»	8
Дефект калибровки, найденный чтением собственного вывода	9
M3 «Пожар» — и главное подтверждение троичной логики	9
Линейная схема	9
Демонстрационные сутки	10
Дальше — День 5	10
День 5 — проникновение, домены, журнал решений	10
Сделано	10
M4 «Проникновение»	10
Залипшие каналы охраны	10
Второй дефект того же класса	10
Карточка получила пятую часть	11
Журнал прогнозов	11
Итог	11
Замечание к доработке карточек	11
Дальше — День 6	11
День 6 — доработка карточек по спецификации	11
Что было заказано	11
Главная сложность — метки времени у признаков	11
Мультикарточка	12
Восемь дефектов, найденных чтением вывода	12

Карточка сейчас	12
Дальше	12
День 7 — роли, настройки, отчёты	13
Сделано	13
Двухфакторная аутентификация	13
Ролевая модель	13
Настройки вместо констант	13
Отчёт XLSX	13
Состояние MVP	14
Интерактивная карта	14
Почему всё-таки сделали	14
Как сделали осмысленной, а не декоративной	14
Честность подписи	14
Состояние MVP	14
Секундная точность меток и каскады	14
Что оказалось в данных	14
Что добавлено	15
Побочный результат: каскады стали видны	15
Критерий	15
Секунды доведены до всех сценариев	15
Третий дефект того же класса	16
Черновик заявки на ремонт	16
Принцип: обоснование не пишется руками	16
Срок и приоритет выводятся из вызревания признаков	16
Исполнитель по сценарию	17
Реализовано	17
Календарные риски и температурные тенденции	17
Календарь	17
Температура: сценарий подмораживания	17
Ещё одна цифра наблюдаемости	18
27 сентября. Согласование числительных и диаграммы	18
Пятьдесят три места, где текст выглядел машинным	18
Диаграммы: чего не было вообще	19
28 сентября. Тесты и что они нашли	19
Почему через браузер	19
Три дефекта, которые нашлись сразу	19
Три дефекта в самих тестах	20
Гонка, которую нельзя лечить паузой	20
Воспроизводимость	20

День 1 — данные, контракт, триты

Сделано

- `core/config.py` — пути, лимиты ОЗУ, периоды-артефакты, сентинелы
- `core/trits.py` — троичная логика Клини: `confirm`, `escalate`, `negate`, наблюдаемость
- `core/contract.py` — семантический контракт, 199 пар значений
- `core/etl.py` — конвейер витрин
- `scripts/01_build_marts.py`, `scripts/02_twins_and_domains.py`

Витрины

Витрина	Строк
dim_channel	11 485 каналов с объектом, пикетом, флагами
fact_channel_day	4 273 083 канало-суток
fact_object_day	124 487 объекто-суток
channel_profile	12 627 профилей активности
logical_domain	3 759 доменов
twin_sync	412 групп тезок

Сборка всех витрин из 313 млн событий — **11 секунд**.

Покрытие контракта — 100 %

313 546 016 событий разложены по классам, непокрытых **ноль**.

Класс	Событий
ТЕЛЕМЕТРИЯ_ЧИСЛО	225 076 402
НОРМА	39 797 930
СРАБОТКА	31 788 907
МУСОР	8 624 153
НЕОПРЕДЕЛЕНО	6 403 315
ОТКАЗ	1 750 304
ОТКЛЮЧЕНО	105 005

Найденные дефекты

Плейсхолдеры в контракте. <ЧИСЛО> и <ДАТА-ВРЕМЯ В ЗНАЧЕНИИ> — шаблоны, а не значения. SQL сравнивал бы `raw_value = '<ЧИСЛО>'`, что не совпадёт никогда. Числа теперь классифицируются приведением и проверкой диапазона, сентинелы (−127, −3276, 999) уходят в мусор.

CASE на 199 условий. На 313 млн строк это сотни строковых сравнений на строку. Заменено на join с таблицей-справочником — один хеш-проход.

Журнал событийный, а не телеметрический

Заполнено лишь **13,6 %** возможных канало-суток (4,27 млн из 31,4 млн). Медиана — 2 события в сутки на канал. Почти ежедневно говорят только 70 каналов (38 событий в сутки) — это газовая телеметрия.

Следствие: отсутствие строки ≠ норма. Для болтливого канала суточное молчание — признак отказа, для молчаливого — обычное дело. Правило «замолчал = отказал» обязано быть персональным, для этого и построен `channel_profile`.

Каналы-тёзки: результат однозначный

Гипотеза заказчика («канал — это показание либо порог») проверена поведенчески.

Вывод	Групп	Пар каналов	Медиана синхронности
Синхронны ≥95 %	386	1 067	1.000
Скорее синхронны 70–95 %	26	85	0.927
Независимы <70 %	0	0	—

Ни одной несинхронной группы. Тёзки ведут себя как один прибор, медиана синхронности ровно 1,0. Значит, их надо схлопывать: иначе модель получит 1 067 дублирующих пар признаков, а диспетчер — несколько одинаковых строк на одно событие.

Логические домены: половина сети слепа

3 759 доменов (объект × пикет):

Размер домена	Доменов
1 канал — подтвердить нечем	1 823
2–3 канала	1 215
4–10 каналов	581
более 10	140

Подтверждение задымления теплом доступно в **135 доменах из 2 140** с дымовыми датчиками:

	Доменов	Каналов
Дым есть, термики нет	2 005	4 563
Дым и термика вместе	135	1 432

Это главный аргумент за троичную логику. В 1 823 точках правило заказчика («тревога засчитывается, если подтверждена соседом») применить не к чему — соседа нет. Бинарная логика обязана свести это к «тревоги нет». Наш `confirm(+1, 0) = 0` возвращает «не знаю», и вердикт честно говорит: подтвердить нечем.

Дальше — День 2

Признаки с шаблонами фраз, таргет M1, обучение и слой объяснимости.

День 2 — признаки, модель M1, объяснимость

Сделано

- `core/labels.py` — таргет «отказ канала в горизонте» на плотном календаре
- `core/features.py` — 20 признаков, каждый с шаблоном фразы
- `core/model.py` — обучение, временной сплит, метрики
- `core/explain.py` — точная декомпозиция вкладов, карточка вердикта
- `scripts/03_train_m1.py`, `scripts/04_horizon_sweep.py`

Таргет

Канало-суток	18 056 743
Позитивов	9 353
Базовая частота	0,0518 %
Каналов в выборке	9 243

Исключено: энергетические типы, агрегаты зон, период внедрения СМБУ-2, сутки обрыва выгрузки, массовые базо-дни (из позитивов).

Контроль фильтра массовых дней сработал. Доля выходных среди позитивов поднялась с 5,74 % до **24,0 %** при календарной норме 28,6 %. То есть след графика бригад из таргета вычищен: до фильтра отказы приходились почти исключительно на рабочее время.

Ошибка, пойманная на проверке баланса

Первая версия отбора дала обучающую выборку 7 541 строка при 3 112 позитивах — почти сбалансированную вместо задуманных 1:100. Делитель был задан константой, а не считался от фактического числа позитивов. При базовой частоте 0,04 % ошибка в делителе на порядок превращает 1:100 в 1:1,4 и ломает калибровку. После исправления — 315 167 строк, precision@100 вырос с 11 % до 16 %.

Объяснимость: обещание выполнено буквально

Декомпозиция вкладов по путям леса проверена на топ-20 вердиктах:

сумма вкладов равна предсказанию: да (максимальное расхождение 4.44e-16)

Это не приблизительное объяснение, а точное: базовая вероятность плюс сумма вкладов в точности равна предсказанию модели. Проверка встроена в обучение и падает, если расхождение превысит допуск.

Что предсказывает модель

Топ признаков по важности:

Признак	Важность
days_since_fault	0,248
fault_ratio_90d	0,132
channel_age_days	0,090
silence_days	0,070
base_failed_share_7d	0,067

Читается осмысленно: канал, недавно отказывавший, отказывает снова; молчание и доля отказавших каналов контроллера — следующие по силе. Последний признак и есть каскад по питающей линии, о котором говорил заказчик.

Горизонт: где проходит граница возможного

Горизонт	Позитивов	База	PR-AUC	P@100	Прирост	Макс. precision
1 сутки	2 401	0,053 %	0,0105	13 %	245×	18 %
3 суток	6 647	0,147 %	0,0091	15 %	102×	17 %
7 суток	14 645	0,324 %	0,0133	12 %	37×	19 %
14 суток	27 906	0,617 %	0,0210	28 %	45×	73 %

Вывод для пояснительной записки. ТЗ требует горизонт не менее 24 часов и Precision 0,7. На этих данных при 24 часах достижимый максимум точности — 18 %. Требуемые 70 % достигаются при горизонте 14 суток. Абсолютная точность на коротком горизонте низка не из-за модели: событие слишком редкое, один отказ на 1 900 каналов-суток, при этом модель работает в 245 раз лучше случайного выбора.

Продуктовое следствие. Два режима вместо одного, что совпадает с делением потока, которое описал сам заказчик:

- **дежурная смена** — горизонт 24 ч, ранжирование очереди, прирост 245×
- **планирование ТО** — горизонт 14 суток, Precision 73 %, соответствует целевому значению ТЗ

Оба горизонта выносятся в настройки админки, как того ожидают организаторы.

Дальше — День 3

Сквозной срез: Docker, PostgreSQL, скелет NiceGUI, REST API. Контрольная точка — `docker compose up` показывает карточку вердикта с обоснованием.

День 3 — сквозной срез

Контрольная точка взята

```
$ docker compose up
Container concorde-db-1   Healthy
Container concorde-app-1 Up (healthy)
```

Приложение отвечает через 6 секунд после старта. Открывается очередь вердиктов, каждый раскрывается в обоснование с вкладом улики.

Сделано

- `core/db.py` — оперативный контур PostgreSQL: вердикты, решения, справочник причин, роли, аудит
- `core/scoring.py` — превращение предсказания в карточку
- `app/main.py` — интерфейс диспетчера и REST API
- `Dockerfile`, `docker-compose.yml`, `deploy/entrypoint.sh`
- `scripts/05_generate_verdicts.py`, `scripts/06_seed_db.py`

Разделение хранилищ

Контур	Где	Что
Аналитический	DuckDB + Parquet	313 млн событий, витрины, признаки
Оперативный	PostgreSQL 16	вердикты, решения, пользователи, настройки, аудит

Требование ТЗ «PostgreSQL 12 и выше» закрывается оперативным контуром. Аналитику держать в Postgres было бы неоправданно: загрузка 313 млн строк съела бы полдня и десятки гигабайт, а DuckDB сканирует их за секунды.

Решение по образу

Витрины и сырые данные в образ **не кладутся**. Приложение читает вердикты только из PostgreSQL, поэтому образ лёгкий, а `docker compose up` поднимается за секунды. Расчёт вердиктов — отдельный офлайн-шаг на машине с данными, результат уезжает в компактный seed-файл (200 вердиктов, 355 КБ).

Это же снимает риск, о котором предупреждал ментор: эксперты могут не суметь развернуть тяжёлый пакет. Здесь разворачивать нечего — два контейнера и одна команда.

Оперативный контур

Справочник причин воспроизводит номенклатуру заказчика, которую он назвал на встрече, но выгрузить отказался: ложная сработка, профилактика, проверка, испытания, подтверждено, выезд, передано в ремонт. Флаг `is_false` отделяет ложные срабатывания — на нём со временем и вырастет та разметка, которой сейчас нет ни у нас, ни у заказчика.

Роли повторяют оргструктуру: техник (один объект) → диспетчер района → диспетчер ОДС, плюс группа реагирования, добавленная по предложению ментора.

Квитирование реализовано буквально по описанию заказчика: неотработанный вердикт выделен и остаётся в ленте, пока диспетчер его не закрыл.

Найденные дефекты

Противоречие в шаблоне фразы. Признак `fault_ratio_90d` выводился как «неисправностей вчетверо больше нормы (×12.2)» — текст спорил с числом. Для продукта, чья ценность в качестве объяснения, это недопустимо. Добавлен слой особых формулировок: «0 суток назад» теперь звучит как «неисправность зафиксирована сегодня».

Пропуски pandas. `NAType` из nullable-колонок не ловится через `numpy.isnan`, из-за чего падало построение подписи места.

Пример вердикта из работающей системы

объект Гамма, ПК1137 — отказ датчика, 97% в ближайшие 24 ч

Почему так решено:

- | | |
|---|--------------|
| 1. сообщений о неисправности за 7 суток: 78 | вклад +10.6% |
| 2. неисправность зафиксирована сегодня | вклад +10.6% |
| 3. неисправностей в 12.2 раза больше личной нормы | вклад +10.3% |
| 4. суток с неисправностью за месяц: 2 | вклад +7.0% |

Это НЕ пожар: сработок дыма за сутки не зафиксировано

Чего мы не видим:

- тепловых датчиков в этой точке нет, подтвердить температурой нельзя

Если устранить главную причину — сообщений о неисправности за 7 суток: 78 — риск снизится с 97% до 78%.

Дальше — День 4

M2 «подтопление» с каскадом по разуклону, M3 «пожар» с отсевом прогонов ТО, линейная схема коллектора по пикетам.

День 4 — подтопление, пожар, линейная схема

Сделано

- `core/pumps.py` — M2: счётчик пусков, личная сезонная норма, каскад по разуклону
- `core/fire.py` — M3: многофакторный детектор с отсевом прогонов ТО
- Вердикты по трём сценариям в общей ленте с фильтром
- Линейная схема коллектора по пикетам

M2 «Подтопление»

Сценарий целиком продиктован заказчиком. Построены витрины:

Витрина	Содержание
<code>pump_day</code>	97 596 канало-суток, 462 353 пуска на 180 насосах
<code>pump_baseline</code>	личная норма на канал × месяц
<code>pump_anomaly</code>	трет отклонения
<code>pump_network</code>	общесетевой контекст

Сезонность воспроизвелась. Суток с общесетевым водопритоком по месяцам: март 31, апрель 19, июнь 18, май 13 — против сентября 1 и февраля 2. Это подпись паводка, и она совпала с тем, что независимо нашёл аудит.

Триты: 73,7 % норма, 22,3 % неизвестно (нормы ещё нет), 4,0 % отклонение.

Дефект калибровки, найденный чтением собственного вывода

Первая версия выдала вердикт: «АНС Н2: 2 пуска за сутки против нормы 1 — риск 99 %».

Причина: у насоса, стабильно делающего один пуск в сутки, MAD близок к нулю, и устойчивый z-score взрывается на разнице в единицу. Формально отклонение огромно, по существу — ничего не произошло.

Такой вердикт уничтожил бы доверие диспетчера быстрее любой пропущенной аварии. Введено требование одновременной значимости — относительной и абсолютной: не менее 10 пусков и не менее 5 сверх личной медианы.

После исправления медианная аномалия — 28 пусков при кратности 6,6×. Пример реального вердикта:

```

объект Альфа, ПК137 — аномальный режим откачки, 95%
• АНС1 Н1 ПК137 Г.1 ПК18 Обр: 27 пусков за сутки против нормы 5
• это в 5 раз выше личной нормы насоса
• наработка за сутки 105 минут
Чего мы не видим: источник воды может быть выше по трассе —
уклон коллектора в данных не задан

```

М3 «Пожар» — и главное подтверждение троичной логики

Показатель	Значение
Сработок дыма	164 288
Термических событий	9 452
Пикето-суток с дымом	50 564
Из них прогоны ТО	36 476 (72 %)

Отсев прогонов ТО убирает почти три четверти материала — без него детектор объявлял бы пожар во всём коллекторе сотни раз.

Распределение трита подтверждения по локальным событиям:

Подтверждение	Пикето-суток	Доля
0 — подтвердить нечем	12 163	86,3 %
-1 — термика молчит	1 910	13,6 %
+1 — подтверждено термикой	15	0,1 %

Это главный эмпирический аргумент всего решения. В 86 % реальных задымлений бинарная система обязана округлить «не могу проверить» до «не подтверждено», то есть до «пожара нет». Правда же в том, что тепловых датчиков в этих точках физически нет. За 8 лет подтверждено теплом всего 15 пикето-суток.

Наш `confirm(дым, термика)` возвращает 0, и вердикт пишет прямо: «тепловых датчиков в этой точке нет — подтвердить задымление нечем; это не значит, что температура в норме».

Линейная схема

Развёртка трассы по пикетам, один пикет — 10 метров. Географических координат в данных нет, и заказчик прямо разрешил их не восстанавливать, предложив именно такое представление: коллектор физически вьётся, но диспетчеру важна не форма трассы, а положение вдоль неё.

Демонстрационные сутки

Выбраны **2026-05-06** — единственные в последнем квартале данных, где присутствуют все три сценария сразу. На случайно взятых сутках подтопление и пожар дают по одному вердикту: события редкие, и это честная характеристика предметной области, а не недостаток модели.

Итог: 151 вердикт — 120 отказов, 20 задымлений, 11 подтоплений.

Дальше — День 5

М4 «проникновение», логические домены сквозным слоем, квитиование и журнал решений.

День 5 — проникновение, домены, журнал решений

Сделано

- `core/intrusion.py` — М4: режим охраны, исправность канала, мультисенсорность, ночное окно
- Рекомендации «что делать» и прецеденты из истории в карточках
- Журнал прогнозов с историей отработки
- Фильтр по четырём сценариям в ленте

М4 «Проникновение»

Метод взят у заказчика дословно: признаком реального проникновения он назвал последовательность сработок разных датчиков, а признаком ложной тревоги — одиночный объёмник в середине коллектора.

Воронка отсева:

Этап	Эпизодов
Всего сработок охранного контура	291 353
Из них при объекте под охраной	10 820
Плюс мультисенсорные	4 493
Плюс ночные	699

Итог — **1,23 сигнала в сутки**. Это рабочая нагрузка диспетчера, а не шквал.

90,4 % всех сработок приходятся на снятый с охраны объект — то есть на легальную работу подрядчиков. Именно этот фильтр и делает сценарий осмысленным.

Залипшие каналы охраны

Из 47 каналов «Состояние охраны» **10 залипли**: медианная доля времени под охраной 0,992 при 199 переключениях за 8 лет. Без этой проверки детектор объявлял бы проникновением любое движение на таких объектах.

Режим охраны выражен тритом, и это принципиально: каналов охраны 47 на 55+ объектов, плюс залипшие. `confirm(сработка, режим)` даёт 0 там, где режим неизвестен, и вердикт пишет «канала состояния охраны на объекте нет — неизвестно, был ли он под охраной», а не «объект не охранялся».

Второй дефект того же класса

Верхним кандидатом на проникновение оказался эпизод с **3 303 срабатываниями за ночь в одной точке**. Человек столько не даёт — это дребезг шлейфа.

Введена верхняя граница правдоподобия: эпизод с числом сработок выше 200 получает нулевой скор и формулировку «для перемещения человека это неправдоподобно много — картина соответствует дребезгу шлейфа; эпизод отнесён к отказам оборудования».

Это тот же класс ошибки, что был у насосов на Дне 4: статистически яркое отклонение, физически бессмысленное. Оба раза дефект нашёлся чтением собственного вывода, а не падением тестов.

Карточка получила пятую часть

Было четыре вопроса: что, почему, что это НЕ, чего мы не видим. Добавлен пятый — **что делать**. Прогноз без действия бесполезен бригаде на объекте.

Тексты рекомендаций заведены таблицей ACTIONS по паре (сценарий, тип датчика), чтобы их правила команда, а не разработчик.

Плюс прецедент из истории: «Похожий случай: 09.12.2025, та же точка, сообщений о неисправности: 1». Заказчик описывает карточку инцидента как документ с историей — прецедент отвечает на вопрос «чем это кончилось в прошлый раз».

Журнал прогнозов

Реестр вердиктов с решениями, счётчики отработанных и признанных ложными. Пары «прогноз — исход» это ровно та разметка, которой сейчас нет ни у нас, ни у заказчика; журнал начинает её копить с первого дня работы.

Итог

166 вердиктов: 120 отказов, 20 задымлений, 15 проникновений, 11 подтоплений. Три страницы: очередь, линейная схема, журнал.

Замечание к доработке карточек

В одном вердикте улики читаются странно: «неисправностей в 12,9 раза больше нормы» рядом с «сообщений о неисправности за 7 суток: 1». Формально верно — личная норма канала меньше одного сообщения в неделю, — но диспетчеру такое сочетание непонятно. Нужна проверка на согласованность улик при доработке карточек.

Дальше — День 6

Карточки первым приоритетом, затем роли, настройки админки и отчёты.

День 6 — доработка карточек по спецификации

Что было заказано

1. «Что произошло» описанием, а не кодом типа
2. Группировка по объектам в мультикарточку — бригада едет на объект целиком
3. Метки времени: появление карточки, появление каждого признака, взятие в работу, закрытие
4. Карточка раскрывается: таймлайн и описания
5. Мультикарточка раскрывается: общий таймлайн, общее описание, вложенные карточки

Главная сложность — метки времени у признаков

Улика обычно является агрегатом по окну: «78 сообщений о неисправности за 7 суток» само по себе метки времени не имеет. Принято определение: **момент появления признака — первые сутки внутри окна, когда условие стало истинным**.

Определение приблизительное по своей природе, поэтому в интерфейсе пишется «признак появился», а не «событие произошло».

Продуктовый смысл меток оказался сильнее ожидаемого. Разница между «признаки копились три недели» и «всё возникло за час» меняет тип реакции: первое закрывается плановой заявкой, второе требует выезда. Появился производный показатель — **вызревание признаков**: медиана 29 суток, максимум 69.

Мультикарточка

166 вердиктов свернулись в **14 мультикарточек** по объектам. Внутри — сортировка по риску, снаружи — по максимальному риску объекта.

объект Гамма, участок ПК906-ПК1137: 10 отказов, 5 проникновений.
Наибольший риск 98% — Прогноз отказа контакта АВ МРСБ АВ ПК906 на объекте Гамма, ПК906.
У 9 из 16 вердиктов ограничена наблюдаемость — подтвердить приборами получится не всё

Общий таймлайн схлопывает одинаковые события разных каналов с указанием точек:

2026-04-08 [признак] суток с неисправностью за месяц: 5 — на 3 точках (ПК1110, ПК1119, ПК1115)
2026-04-30 [признак] Н1 ПК1050: 179 пусков за сутки против нормы 130 — ПК1050

Восемь дефектов, найденных чтением вывода

Доработка карточек оказалась в первую очередь работой над качеством текста. Каждый дефект найден чтением собственного вывода, а не падением теста.

Дефект	Как выглядел	Исправление
Кратность без опоры	«неисправностей в 12,9 раза больше нормы» при «сообщений за 7 суток: 1»	таблица MIN_SUPPORT : относительная улика требует абсолютной опоры
Удвоение слова	«на объекте объект Гамма»	склонение названия вместо конкатенации
Регистр ломает обозначения	«контакта ав мрсб ав пк906»	заголовок не приводится к нижнему регистру
Дубли в таймлайне	одна фраза трижды подряд	схлопывание с указанием точек
Метка из будущего	признак «с 2026-05-07» при вердикте 06.05	молчание не может начаться завтра
Мнимая аномалия насоса	«179 пусков против нормы 130»	относительный порог 1,5× наряду с абсолютным
Бессмысленная кратность	«это в 1 раз выше нормы»	порог отображения 1,5×
Неправдоподобное проникновение	3 303 сработки за ночь	граница правдоподобия для человека

Три из восьми — один и тот же класс ошибки: **статистически яркое отклонение, физически бессмысленное**. Относительная мера без абсолютной опоры даёт формально верные и практически абсурдные утверждения. Для решения, которое продаёт объяснимость, это опаснее пропущенной аварии: один такой вердикт обесценивает все остальные.

Карточка сейчас

Семь частей: что произошло · почему (с вкладами и метками) · что это НЕ · чего мы не видим · что делать · похожий случай · хронология.

Дальше

Роли и RBAC, настройки админки, отчёты XLSX. Затем сборка пакета и видео.

День 7 — роли, настройки, отчёты

Сделано

- `core/auth.py` — вход с двухфакторной аутентификацией, ролевая модель, область видимости
- `core/reports.py` — выгрузка XLSX для руководства
- Страница входа, админка настроек, журнал действий
- Разграничение доступа в ленте вердиктов

Двухфакторная аутентификация

Ментор назвал её примером требования, невыполнение которого без объяснения снимает решение по формальному признаку. Поэтому она работающая, а не задекларированная: TOTP с окном в один шаг, проверено на верном и неверном коде.

Интеграция с LDAP/AD оставлена обозначенной точкой расширения: подключение к каталогу заказчика вне контура конкурса, и это зафиксировано в таблице отклонений.

Ролевая модель

Роль	Область	Решения	Настройки
Техник	один объект	нет	нет
Диспетчер района	группа объектов	да	нет
Диспетчер ОДС	всё предприятие	да	да
Группа реагирования	объект	да	нет

Область видимости задаётся таблицей `user_object`, а не ролью напрямую — назначения меняются без правки кода.

Настройки вместо констант

Организаторы ожидают, что пороги качества и горизонт прогноза задаются администратором: «у администратора есть возможность поправить». В админке редактируются Precision, Recall, горизонт, размер очереди и показ топонимов. Каждое изменение пишется в журнал действий — требование ТЗ по журналированию.

Отчёт XLSX

Три листа: вердикты, сводка, наблюдаемость.

Третий лист оказался самым интересным. Он отвечает на вопрос, которого заказчик себе не задавал: **какую долю вердиктов невозможно подтвердить приборами.**

Объект	Вердиктов	Из них слепых	Доля
объект Кси	48	45	94 %
объект Бета	26	25	96 %
ПС объект Ро	13	12	92 %
объект Эпсилон	29	22	76 %
объект Гамма	15	8	53 %

Это прямое основание для заявки на дооснащение, посчитанное из данных. Побочный результат троичной логики: система, которая честно отличает «норму» от «не знаю», умеет посчитать собственную слепоту.

Состояние MVP

Из 29 пунктов границ MVP закрыты 27. Остаются сборка пакета документации и видеодемонстрация — работа команды.

Интерактивная карта

Почему всё-таки сделали

Линейная схема закрывала задачу ориентирования, но формально оставляла невыполненным пункт ТЗ «интерактивная карта: отображение объектов инфраструктуры с выделением проблемных зон». По правилу конкурса необъяснённое невыполнение снимает решение, а объяснить отсутствие карты было бы нечем: заказчик на встрече не запрещал её, а наоборот снял ограничения.

Дословно [00:41:45–00:43:29]: внутренняя система координат «требуется пересчёта в Яндекс.Карты, там был 2ГИС, неважно», пересчёт «за рамками проекта», а дальше — «возвращаясь к визуализации, здесь никаких абсолютно ограничений нет. Просто сгенерировать линейные объекты». Упомянуты Leaflet и OpenLayers.

То есть Яндекс и 2ГИС прозвучали, но как то, во что пришлось бы пересчитывать их координаты, а не как требование к нам.

Как сделали осмысленной, а не декоративной

Случайная геометрия закрыла бы требование формально и была бы бесполезна. Вместо этого использованы **реальные топонимы, уцелевшие в названиях датчиков**: Зюзино, Ясенево, Беляево, Коньково, Коммунарка, Донской, Гагаринский, Академическая, Новочерёмушкинская, Никулино, Зелинского, Профсоюзная.

12 объектов из 14 привязаны к своим районам. Длина трассы считается из пикетов: $(ПК_{\max} - ПК_{\min}) \times 10$ метров. Суммарно 60,4 км, 156 точек вердиктов.

Объект	Район	Участок	Длина	Вердиктов
объект Гамма	Ленинский	ПК0–ПК1195	11,9 км	15
объект Кси	Коммунарка	ПК0–ПК302	3,0 км	48
объект Тау	Академическая	ПК0–ПК602	6,0 км	11
объект Бета	Донской	ПК0–ПК106	1,1 км	26

Честность подписи

Карта сопровождается прямой оговоркой: геометрия схематическая, реальные координаты во внутренней системе и в датасет не передавались, расположение восстановлено по районам, длина — по пикетам. «Для ориентирования этого достаточно, для съёмочных задач — нет».

Это тот же принцип, что и в вердиктах: показывать то, что знаем, и явно называть границу знания.

Состояние MVP

Закрыто 28 пунктов из 29. Остаётся сборка пояснительной записки и видео.

Секундная точность меток и каскады

Что оказалось в данных

Журнал хранит время **с точностью до секунды**: из 30 695 389 событий 2026 года 30 089 430 имеют ненулевые секунды, встречаются все 86 400 времён суток. Газовая телеметрия пишет каждые 5 секунд.

То есть ограничение было целиком нашим: витрина агрегировала до суток. `fact_channel_day` хранил `t_first` и `t_last`, но по всем событиям сразу, без разделения по классам.

Что добавлено

В витрину добавлены метки по семантическим классам: `t_first_fault`, `t_last_fault`, `t_first_trigger`, `t_first_undefined`. Пересборка — 12 секунд.

Метка признака теперь указывает момент, а не сутки:

```
2026-05-06 14:45:35 сообщений о неисправности за 7 суток: 1
2026-05-06 14:45:35 неисправность зафиксирована сегодня
2026-05-06 14:47:24 канал выходил на связь сегодня
```

Там, где момент неизвестен, метка остаётся суточной: подменять её серединой суток было бы вымыслом.

Побочный результат: каскады стали видны

Секунды вскрыли то, что посуточная агрегация скрывала полностью. Заказчик описывал физику на встрече: информационно-питающая линия 48 В кормит десятки датчиков, и её повреждение роняет их разом.

В данных это выглядит так:

Объект	Каскад
объект Кси	39 признаков на 22 точках за 17 с , отрезок ПК222–ПК276 (540 м)
объект Бета	36 признаков на 13 точках за 19 с, отрезок ПК2–ПК91 (890 м)
объект Гамма	23 признака на 7 точках за 54 с, отрезок ПК1110–ПК1137 (270 м)
объект Тау	8 признаков на 4 точках за 1 секунду , отрезок ПК296–ПК354

Каскады найдены на 7 объектах из 14.

Различие принципиальное для бригады: десять независимых отказов требуют десяти проверок, один каскад — одной, на питающей линии. Без секундных меток эти события выглядели бы как несвязанные отказы, случившиеся «в один день».

Критерий

Первая версия давала ложные срабатывания вида «3 признака за 0 с» в одной точке. Совпадение признаков на одном приборе — это не каскад по лучу, и называть его так значило бы отправить бригаду не туда. Критерий ужесточён: не менее четырёх признаков **минимум на двух разных пикетах** в окне 120 секунд. После этого осталось 7 объектов из 14 — те, где отрезок действительно протяжённый.

Секунды доведены до всех сценариев

Эпизодные сценарии (пожар, проникновение, подтопление) собирали метку строкой из даты и времени, а дата приходит из pandas как `Timestamp`. Получалось `2026-05-06 00:00:00T10:46:30` — невалидная ISO-строка, которую интерфейс обрезал до полуночи. Сборка переведена на `datetime.combine`.

Дополнительно у эпизодов появилась продолжительность, а у насосов — окно откачки:

```
подтопление 00:29:09 АНС1 Н1 ПК137: 27 пусков за сутки против нормы 5
              00:29:09 откачка шла с 00:19:38 до 23:07:29
пожар        10:02:46 в одной точке сработало 11 независимых дымовых датчиков
              10:02:46 эпизод длился 19 мин: с 10:02:46 до 10:22:04
```

Покрывание: 469 меток из 469 (100 %) с секундной точностью.

Третий дефект того же класса

Продолжительность эпизода немедленно вскрыла ошибку в М4: вердикт «возможное проникновение, 84 срабатывания» имел эпизод длиной **12 часов** (10:46–22:44). Человек не проникает на объект двенадцать часов — это работа подрядчиков при формально не снятой охране либо залипший датчик.

Критерии мультисенсорности и режима охраны это пропускали, потому что не смотрели на длительность. В посуточном представлении «84 срабатывания» и «84 срабатывания за 12 часов» вообще неразличимы — дефект стал виден только после перехода на секунды.

Введена граница правдоподобия: эпизод дольше 4 часов получает нулевой скор с объяснением. Вердиктов проникновения стало 13 вместо 15.

Это третий случай одной и той же ошибки за проект: **статистически заметное отклонение, физически бессмысленное**. Первый — насос с кратностью от нулевого разброса, второй — 3 303 срабатывания за ночь, третий — двенадцатичасовое проникновение. Все три найдены чтением собственного вывода. Вывод для метода: каждому правилу нужна проверка на физическую правдоподобность, а не только на статистическую значимость.

Черновик заявки на ремонт

Закрывает пробел, вскрытый таблицей ответов: ответ 20 и ТЗ называют черновик заявки явно, а у нас была только фиксация решения диспетчера.

Принцип: обоснование не пишется руками

Поля заявки выводятся из вердикта, а обоснование переносится из карточки целиком — с теми же числами и метками времени:

Проверка контакта АВ — объект Гамма, ПК906.
 Приоритет высокий, срок до 09.05.2026, исполнитель: Служба эксплуатации КИПиА.

ОБОСНОВАНИЕ:

2026-05-06 14:45:35 — сообщений о неисправности за 7 суток: 1
 2026-05-06 14:45:35 — неисправность зафиксирована сегодня
 2026-05-06 14:47:24 — канал выходил на связь сегодня

Похожий случай: 09.12.2025, та же точка, 1 сообщений о неисправности.

Если бы диспетчер набирал обоснование заново, заявка со временем начала бы расходиться с данными, на которых построен прогноз, и потеряла бы доказательную силу. Редактировать можно тип работ, приоритет, срок и исполнителя — то есть решения, а не факты.

Срок и приоритет выводятся из вызревания признаков

Показатель вызревания, появившийся при переходе на метки времени, здесь получил прикладной смысл. Медиана по данным — 29 суток.

Условие	Приоритет	Срок
каскад по питающей линии	аварийный	+1 день
пожар или проникновение при риске $\geq 0,6$	аварийный	+1 день
риск $\geq 0,85$	высокий	+3 дня
риск $\geq 0,6$, вызревание > 7 суток	средний	+10 дней
риск $\geq 0,6$, развитие внезапное	высокий	+3 дня
остальное	низкий	+30 дней

То, что копилось месяц, не обязано чиниться завтра; то, что возникло за час, не может ждать планового обхода. Каскад повышает приоритет независимо от вероятности: отказ луча обесточивает десятки точек сразу.

Исполнитель по сценарию

Отказы — служба эксплуатации КИПиА, подтопление — служба водоудаления, пожар — дежурная бригада, проникновение — группа реагирования. Таблицей, чтобы формулировки правила эксплуатации, а не разработчик.

Реализовано

- таблица `work_order_draft` в оперативном контуре
- кнопка «Заявка» на карточке вердикта, диалог с проверкой полей
- защита от дублей: если заявка по вердикту уже есть, это показывается
- страница реестра заявок со сводкой по приоритетам
- четвёртый лист «Заявки» в выгрузке XLSX

Реальная отправка во внешнюю систему не требуется — журнал ОДС ведётся отдельно на Django и интегрирован не будет, что организаторы подтвердили письменно.

Календарные риски и температурные тенденции

Оба фактора названы организаторами среди ожидаемого содержания карточки прогноза.

Календарь

Заказчик сформулировал механику на встрече: нарушители непрогнозируемы, «но есть вполне характерные календарные точки — праздничные дни, большие длинные праздничные дни, есть особые праздники, например, 9 мая».

Механика двойная, и в этом суть: в праздники объект чаще остаётся без присмотра, а бригад на линии меньше, то есть растёт и риск, и время реакции.

Дата	Множитель	Формулировка
9 мая	×1,35	объекты без постоянного присмотра, бригад меньше обычного
Новогодние каникулы	×1,30	8-й день длинных выходных
Обычный выходной	×1,10	плановых работ на объекте не ведётся

Срок заявки сдвигается на день перед началом длинных выходных: работа, срок которой приходится на нерабочий период, провисит все выходные.

Ограничение проговаривается честно: используется фиксированный набор нерабочих дней из Трудового кодекса. Ежегодные переносы, утверждаемые постановлением Правительства, не учитываются — официального производственного календаря в данных нет.

Температура: сценарий подмораживания

Реализован сценарий, которого нет в ТЗ, но который заказчик описал дословно: сорванный люк выпускает холодный воздух, и возникает угроза подмораживания трубопроводов холодной воды. Метод он же и назвал — «смотреть соседние пикеты, куда дальше эта температура пошла».

Построены витрины: суточная температура по точкам (289 208 точка-суток), сезонная норма точки (4 982), обычный отрыв от соседей (412).

Найденный дефект. Первая версия сравнивала точку с соседями по абсолютному порогу в 4 °С. Признак срабатывал равномерно круглый год — 2 700–4 600 случаев в любом месяце, то есть ловил геометрию, а не происшествия: пикет у вентилях холоднее соседей всегда.

После перевода на собственную базу точки (обычный для неё отрыв) появилась зимняя сезонность, которой и ждёшь от нарушения контура:

Месяц	Аномальный отрыв
январь	3 706
декабрь	3 066
февраль	3 101
июль	663
август	625

Соотношение зима/лето почти шестикратное. Доля отклонений упала с 19 % до 10,5 %.

Это четвёртый случай одной и той же ошибки за проект: относительная мера без собственной базы измеряет устройство системы, а не событие в ней.

Ещё одна цифра наблюдаемости

Температура есть лишь в **416 точках из 3 774**. У 2 523 точек (67 %) температурных датчиков нет нигде на объекте. Поиск расширен до ближайшего датчика в пределах 100 метров — это добавляет 674 точки, и расстояние указывается в улике. Дальше расширять нельзя: тепловой режим меняется вдоль трассы, и показание за 300 метров о состоянии точки уже ничего не говорит.

Где температуры нет, карточка говорит об этом прямо. Сейчас таких карточек 31 из 162.

27 сентября. Согласование числительных и диаграммы

Пятьдесят три места, где текст выглядел машинным

Проверка набора вердиктов регулярными выражениями дала 53 совпадения: 33 раза «1 сообщений о не-исправности», 18 раз «в 2 раз выше личной нормы», дважды «1 суток назад». Формально это мелочь. Практически — это ровно тот признак, по которому читатель понимает, что текст собрала машина, и начинает не доверять остальному в карточке.

Причина одна и та же везде: число подставлялось в готовую строку, и форма существительного была выбрана под множественное число, потому что так чаще.

В проекте уже была функция склонения — в `core/cards.py`, для фразы «2 задымления, 1 отказ» в сводке мультикарточки. Она работала, но применялась в одном месте из четырнадцати. Это типичная история: правило есть, но живёт там, где его однажды написали.

Правило вынесено в `core/plural.py` и подставляется прямо в шаблон фразы:

```
"phrase": "канал выходил на связь {v:.0f} {сутки|суток|суток} из 7"
```

Так автор текста видит все три формы рядом, когда пишет фразу, и не может забыть про одну. Обычные поля формата (`{v:.0f}`, `{base}`) подстановка не трогает: в них нет вертикальной черты.

Исправлено четырнадцать мест в пяти модулях, вердикты пересобраны, проверка даёт ноль совпадений. Локальная копия правила из `cards.py` удалена.

Диаграммы: чего не было вообще

Критерий финальной экспертизы звучит дословно: «проверка объективности диаграмм (оценка достоверности диаграмм будет проводиться на основании их визуальной восприимчивости и отражения данных)». В интерфейсе были линейная схема и карта, но ни одного графика — то есть по этому пункту мы получали ноль при полностью работающем продукте.

Сделаны шесть диаграмм: страница `/analytics` и диаграмма вкладов улик внутри карточки вердикта. Три решения по ходу стоит записать, потому что каждое из них — про достоверность, а не про оформление.

Палитра посчитана, а не подобрана. Четыре цвета сценариев проверены на различимость при дейтеранопии и тританопии. Первый вариант — жёлтый, синий, красный, фиолетовый — прошёл: худшая пара даёт ΔE 13,0 при пороге 8 и 16,3 для нормального зрения при пороге 15. Второй вариант, где отказ рисовался оранжевым, провалился с ΔE 5,6: оранжевый и красный для дальтоника почти неразличимы. На глаз эта пара выглядела нормально.

Заодно убран разнобой: цвет сценария задавался в четырёх местах — в ленте, на схеме, на карте и в легенде, — и «проникновение» было фиолетовым в одном месте и серым в другом. Теперь источник один, `core/charts.py`.

Ось полноты сделана числовой. Первая версия кривой «цена полноты» использовала категориальную ось: 10 %, 20 %, 30 %, 50 %, 70 % стояли на равных расстояниях. На такой оси участок от 30 % до 50 % выглядит таким же, как от 10 % до 20 %, и кривая врёт ровно о том, ради чего построена, — о скорости роста нагрузки. Ось переделана в числовую, и форма кривой изменилась заметно.

Двух шкал на одном поле нет нигде. Точность и число проверок в сутки просятся на один график и дали бы красивую картинку. Это тот самый случай, когда выравнивание двух шкал произвольно и создаёт корреляцию, которой в данных нет. Точность ушла в подсказку при наведении и в таблицу под графиком.

Диаграмма вкладов строится при раскрытии карточки, а не заранее: в очереди до двух сотен карточек, и двести графиков в DOM подвешивают браузер ради того, чего никто не открывает. Страница очереди после этого весит 1,36 МБ вместо 1,6 МБ, а графики появляются там, где на них смотрят.

28 сентября. Тесты и что они нашли

Почему через браузер

Интерфейс живёт на веб-сокетах: страница дособируется после загрузки, содержимое раскрывающихся блоков строится только по клику. Проверять его запросом к HTTP бессмысленно — половины разметки в ответе сервера просто нет, и тест на текст проходил бы при полностью сломанном экране.

Поэтому Selenium и headless Chrome. Тесты ходят тем же путём, что человек: выбирают пользователя в списке, читают одноразовый код со страницы входа, раскрывают блоки, нажимают кнопки. Получилось 108 тестов: 53 на чистую логику за 0,07 секунды и 55 через браузер за две с половиной минуты.

Три дефекта, которые нашлись сразу

Содержимое мультикарточки рисовалось мимо блока. Раскрывающиеся блоки создавались без контекстного менеджера:

```
timeline = ui.expansion("Общая хронология объекта") # без with
lazy_content(timeline, draw_timeline)                # слот уходит наружу
```

Место под ленивое содержимое оказывалось не внутри блока, а рядом с ним. Внешне почти правильно — карточки появлялись, — но оставались видимыми после закрытия блока. Глазами это выглядит как «ну, открылось»; проверка «блок раскрылся и в нём есть текст» поймала за секунду.

Замечу отдельно: дефект появился вместе с ленивой отрисовкой, то есть с исправлением производительности. Одно исправление внесло другую ошибку, и без тестов она уехала бы в поставку.

Вердикты не отдавали идентификатор объекта. Из-за этого разграничение области видимости сравнивало `None` с назначениями, и техник получал пустой экран вместо своего участка. Нашлось при подготовке руководства пользователя, теперь закрыто тестом API.

Право фиксации решений не проверялось. Тоже нашлось при подготовке руководства. Теперь на это есть тест, который не даст ошибке вернуться.

Три дефекта в самих тестах

Честности ради: столько же ошибок было в тестах, и все три одного рода — неверное представление о том, как устроен Quasar.

Поиск поля по `input[aria-label=...]` не работает для выпадающего списка: внутри лежит скрытый `input` размером 1×1 пиксель, а видимую часть рисует соседний элемент. Кнопка с иконкой содержит в тексте ещё и имя иконки — `check\нКВИТИРОВАТЬ`, — поэтому сравнение на равенство её не находит. А поиск блока по всему его тексту находил внешнюю мультикарточку вместо вложенного блока, и клик закрывал родителя вместо раскрытия ребёнка.

Вывод, который стоит записать: тест на интерфейс проверяет не только приложение, но и понимание разработчиком того, как приложение устроено. Оба вида ошибок выглядят одинаково — тест красный, — и различать их приходится каждый раз заново.

Гонка, которую нельзя лечить паузой

Вход изредка не проходил: значение поля уезжает на сервер по веб-сокету, нажатие кнопки — отдельным сообщением, и клик иногда обгоняет синхронизацию. Сервер видит пустой код.

Пауза перед кликом здесь плохое решение: маленькая не спасает, большая замедляет весь набор. Сделано иначе — неудачная попытка распознаётся по сообщению на экране, и вход повторяется со свежим кодом. В наборе нет ни одной паузы фиксированной длины, и это не украшение: тест, который «обычно проходит», приучает не доверять результату.

Воспроизводимость

Сценарные тесты закрывают вердикты и создают заявки — иначе замкнутый цикл не проверить. Первый прогон это заметил не сразу: тесты брали первую карточку, а она уже была отработана предыдущим прогоном, и падали не потому, что в приложении ошибка.

Теперь перед сценарными тестами стенд возвращается к исходному набору теми же двумя командами, что описаны в инструкции. Если сброс недоступен, тесты ищут неотработанный вердикт вместо первого попавшегося.